L42 — Hash Tables and Hash Functions

Unit: 3 | **Chapter:** Hashing | **BT Level:** BT3 | **CO:** CO2

Learning Objectives

- Explain the concept of hashing and hash tables
- Design and evaluate good hash functions
- Analyze expected time complexity of hash table operations
- Implement a basic hash table in Python

---

1. Motivation

Problem: Store and retrieve data by key in O(1) time.

- Array: O(1) access by index, but index must be an integer 0..n-1
- Linked list: O(n) search
- BST: O(log n) search

Hash table: Maps arbitrary keys (strings, numbers, objects) to array indices using a **hash function** → average O(1) for all operations.

---

2. Hash Table Concept

```
Key Space        Hash Function          Array (Table)
"Alice"    →         h("Alice") = 2  →   index 2
"Bob"      →         h("Bob")   = 5  →   index 5
"Charlie"  →         h("Charlie") = 2 → index 2 (COLLISION!)

Table: [_, _, "Alice"/"Charlie", _, _, "Bob", ...]
```

Collision: Two different keys map to the same index. Must be handled (see L43).

---

3. Hash Function Properties

A **good hash function** should:

1. Be **deterministic:** same key always gives same hash
2. Be **fast to compute:** O(1) or O(key length)
3. Be **uniform:** distribute keys evenly across all slots
4. **Minimize collisions:** different keys should hash to different slots

---

4. Common Hash Functions

4.1 Division Method

$$h(k) = k \mod m$$

- m = table size; best when m is a prime not close to a power of 2

```
def hash_division(key, m):
    return key % m
```

4.2 Multiplication Method

$$h(k) = \lfloor m \cdot (k \cdot A \mod 1) \rfloor$$ where A ≈ (√5 - 1)/2 ≈ 0.618 (Knuth's constant)

```
import math

def hash_multiplication(key, m, A=0.618033988749895):
    return int(m * ((key * A) % 1))
```

4.3 String Hashing (Polynomial Rolling)

```
def hash_string(s, m, base=31):
    """Polynomial rolling hash. O(|s|)"""
    h = 0
    for char in s:
        h = (h * base + ord(char)) % m
    return h
```

Better with a large prime modulus to minimize collisions:

```
def hash_string_robust(s):
    """Robust string hash using large prime modulus."""
    MOD = (1 << 61) - 1   # Mersenne prime
    BASE = 131
    h = 0
    for char in s:
        h = (h * BASE + ord(char)) % MOD
    return h
```

4.4 Python's Built-in Hash

```
hash("hello")       # Returns integer (varies per Python session)
hash(42)            # 42
hash((1, 2, 3))     # Tuple hash
hash([1, 2])        # TypeError: unhashable type: list
```

Python uses SipHash (with random seed per process for security).

---

5. Hash Table Implementation

```
class HashTable:
    def __init__(self, capacity=16):
```

```
        self.capacity = capacity
        self.size = 0
        self.buckets = [[] for _ in range(capacity)]   # Chaining

    def _hash(self, key):
        return hash(key) % self.capacity

    def put(self, key, value):
        """Insert or update key-value pair. Average O(1)"""
        idx = self._hash(key)
        bucket = self.buckets[idx]

        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)   # Update existing
                return

        bucket.append((key, value))        # New entry
        self.size += 1
        if self.size / self.capacity > 0.75:
            self._resize()

    def get(self, key):
        """Retrieve value for key. Average O(1)"""
        idx = self._hash(key)
        for k, v in self.buckets[idx]:
            if k == key:
                return v
        raise KeyError(key)

    def delete(self, key):
        """Delete key. Average O(1)"""
        idx = self._hash(key)
        bucket = self.buckets[idx]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket.pop(i)
                self.size -= 1
                return
        raise KeyError(key)

    def _resize(self):
        """Double capacity when load factor exceeds 0.75. O(n)"""
        old_buckets = self.buckets
        self.capacity *= 2
        self.size = 0
        self.buckets = [[] for _ in range(self.capacity)]

        for bucket in old_buckets:
            for key, value in bucket:
                self.put(key, value)

    def __contains__(self, key):
```

```
        idx = self._hash(key)
        return any(k == key for k, _ in self.buckets[idx])
```

---

6. Load Factor

$$\alpha = \frac{n}{m}$$

where n = number of entries, m = number of slots.

- **$\alpha < 0.75$:** Good performance (Python dict target)
- **$\alpha > 1$:** Many collisions, degraded performance
- When α exceeds threshold → **rehash** (double capacity, reinsert all)

---

7. Expected Time Complexity

Operation	Average Case	Worst Case
Search	$O(1 + \alpha) \approx O(1)$	O(n) — all keys collide
Insert	O(1) amortized	O(n)
Delete	$O(1 + \alpha) \approx O(1)$	O(n)
Space	O(n + m)	O(n + m)

With good hash function and α kept < constant → **O(1) expected**.

---

8. Applications

Application	Hash Table Use
Python dict, set	Key-value store, membership test
Database indexing	Hash index for equality queries
Compiler symbol table	Variable name → memory address
Cache (LRU Cache)	Key → cached value
Deduplication	Detect duplicate records
Frequency counting	Count occurrences of elements
Cryptographic hashing	MD5, SHA-256 (for data integrity, not tables)

---

9. Python dict and set

Python's built-in dict and set are backed by a highly optimized hash table:

```
# Dict operations are O(1) average
d = {}
d['key'] = 'value'    # O(1) insert
x = d['key']          # O(1) lookup
del d['key']          # O(1) delete
'key' in d            # O(1) membership
```

```
# Set for deduplication
seen = set()
unique = [x for x in items if not (x in seen or seen.add(x))]

# Frequency count
from collections import Counter
freq = Counter("abracadabra")
print(freq)   # {'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1}
```

Summary

- Hash table: maps keys to array indices via a **hash function** → O(1) average operations.
- Good hash function: uniform distribution, fast, deterministic.
- **Load factor $\alpha = n/m$:** keep below 0.75; rehash when exceeded.
- Collisions are inevitable — handled by chaining (L43) or open addressing (L43).
- Python's `dict` and `set` are practical hash table implementations.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 11 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.7 (Springer, 2020)